Card Fraud Triage Using a Gaussian Mixture Model Phase 1 Report

H Maheshwari, Vaibhav, Mayank, Keshav, Mitul Bhatia
Team T011, Section A, Project 26
Newton School of Technology, Rishihood University
2401010174, 2401010486, 2401010267, 2401010222, 2401010279

***Abstract*—We build a simple card-fraud review system using a Gaussian Mixture Model (GMM). The model learns patterns from earlier legitimate transactions and assigns larger anomaly scores to less typical later transactions. We compare four component counts using training AIC and BIC, choose alert cutoffs on validation data, and evaluate on a later mixed test period. The selected eight-component GMM achieves trapezoidal precision-recall area 0.1496. Our default validation policy finds 37 of 74 test frauds and falsely alerts on 172 legitimate transactions. Reviewing exactly the top 100 scores finds 29 frauds. This report covers Phase 1 machine learning only.**

***Index Terms*—fraud triage, Gaussian mixture, anomaly detection, precision-recall**

I. The Problem

Fraud triage means deciding which transactions a person should review first. We cannot inspect every transaction. Our question is: can a model of legitimate activity place fraud near the front of a review queue? Unusual activity is a reason to investigate, not proof of fraud. The course specifies machine learning for Phase 1, so we implement the GMM. The autoencoder remains Phase 2 work.

II. Literature Review

Dempster, Laird and Rubin [1] explain expectation-maximization (EM), which alternates estimating hidden memberships and updating parameters. This explains how our GMM learns its components. Saito and Rehmsmeier [2] explain why precision-recall measures reveal the burden of false alerts when positives are rare. We therefore emphasize PR area, recall, precision and review workload.

The scikit-learn documentation [3] describes GMM settings and information-criterion selection. We use its diagonal-covariance GMM and a small K grid. The recommended data comes from ULB/Worldline [4]. TensorFlow documents the public CSV mirror [5]. These sources explain our design; our result numbers come from our own run.

III. Data Quality and Exploration

The original CSV has 284,807 transactions, 31 columns and 492 frauds. Time records elapsed seconds. V1–V28 are anonymous numerical inputs transformed by the provider. Amount is in EUR. Class gives the correct answer: zero for legitimate and one for fraud. We do not know the business meanings of the V columns or how the provider fitted its original PCA.

TABLE I
Earlier training and later evaluation periods

Set	Transactions	Legitimate	Fraud
Training	170,235	169,893	342
Validation	56,745	56,688	57
Test	56,746	56,672	74

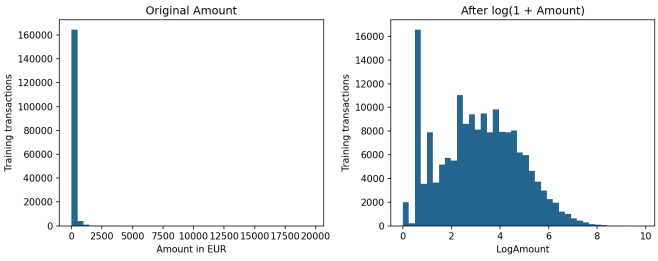


Fig. 1. The log transform compresses the large range of Amount. Both panels use training rows only.

We find no missing or infinite cells. We remove 1,081 exact repeated rows and keep the first occurrence. Nineteen removed rows carry fraud labels, leaving 283,726 transactions and 473 frauds. There is no transaction ID, so duplicate removal is an explicit assumption.

We split earlier and later transactions into approximately 60% training, 20% validation and 20% test. Equal timestamps stay together. Table I shows the counts. The scaler and GMM use only the 169,893 legitimate training rows. Validation and test retain both classes. Test labels do not choose inputs, K or cutoffs.

The notebook includes five initial charts: split sizes, training class counts, amount distributions, hourly transaction counts, and V1/V2 overlap. Fraud is rare, amounts have a long right tail, and the two classes overlap. A policy predicting all test rows legitimate would have about 99.87% accuracy but catch no fraud. This is why we do not use accuracy as the headline result.

IV. Method, Explained Step by Step

A. Prepare the Inputs

We use 29 inputs: V1–V28 and LogAmount, where

$$\text{LogAmount} = \log(1 + \text{Amount}). \tag{1}$$

The transform accepts zero and compresses large amounts without removing them. We exclude Class because it is the answer. Time determines the split and does not enter the model.

StandardScaler applies $(x - \mu)/\sigma$ to each input, where μ and σ are its mean and standard deviation among legitimate training rows. We fit these numbers once and apply them unchanged to validation and test. Learning scaling values from test data would leak future information into preparation.

B. Learn Legitimate Patterns with a GMM

A Gaussian describes a bell-shaped pattern with a center and spread. A GMM combines K such patterns:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{2}$$

Here π_k is a component's weight, μ_k its center and Σ_k its spread/covariance. The weights sum to one. Components are mathematical patterns, not verified customer categories.

EM repeats two steps. The E-step estimates each transaction's membership in each component. The M-step updates weights, centers and spreads using those estimates. We use diagonal covariance, which learns separate input variances within each component and misses some input relationships. We use three initializations, seed 42, up to 200 iterations, and a small variance addition 10^{-6} to avoid a spread collapsing to zero. All four candidates reach their stopping rule; this does not prove a global optimum.

Labels select known legitimate training rows, but GMM fitting receives only inputs and no target labels. Both facts matter when describing the training setup.

C. Choose the Component Count

We try K=1,2,4,8 on the same legitimate rows. AIC and BIC balance a better fit against extra adjustable parameters:

$$\text{AIC} = 2q - 2L, \qquad \text{BIC} = q \log N - 2L. \tag{3}$$

N is the number of fitted rows, q is the number of free parameters, and L is total training log likelihood. Lower values are preferred. We select minimum training BIC and report AIC alongside it. Eight wins within this grid (Fig. 2). It is the largest K tried, so we do not claim it is best among every possible value. Better legitimate density fit does not guarantee better fraud recall.

D. Score Transactions and Set Cutoffs

We use anomaly score $s(x) = -\log p(x)$. Low legitimate density produces a larger score. The score is not a probability of fraud. The code uses the negative of `score_samples`; the GMM's `predict` method would instead return component IDs.

We create an alert when the score is strictly above a cutoff. False-positive rate (FPR) means legitimate transactions alerted divided by all legitimate transactions. We predefine validation FPR budgets of 0.1%, 0.5% and 1%. The default 0.5% is a classroom example fixed before test evaluation. Its cutoff is the 99.5th percentile of legitimate validation scores, using the conservative `higher` percentile method. This gives threshold 75.8942. Later test FPR may differ, so we carry this cutoff unchanged into test.

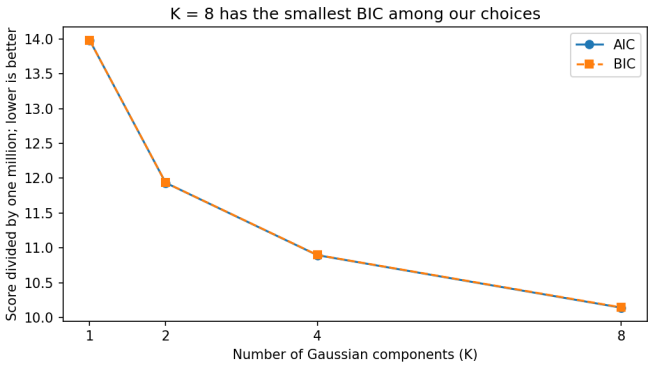


Fig. 2. Eight components give the smallest BIC among our four choices. The curves remain decreasing at the grid boundary.

TABLE II
TEST RESULTS AT FIXED VALIDATION BUDGETS

Budget (%)	Fraud found	False alerts	Recall (%)	Precision (%)
0.1	6	39	8.11	13.33
0.5	37	172	50.00	17.70
1.0	52	381	70.27	12.01

V. Results and Their Meaning

At the default cutoff, the test confusion counts are TN=56,500, FP=172, FN=37 and TP=37. TN means legitimate without an alert; FP means a legitimate false alert; FN means missed fraud; TP means fraud found. Therefore:

$$\text{recall} = \frac{TP}{TP + FN} = \frac{37}{74} = 50\%, \tag{4}$$

$$\text{precision} = \frac{TP}{TP + FP} = \frac{37}{209} = 17.70\%, \tag{5}$$

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{172}{56{,}672} = 0.3035\%. \tag{6}$$

The workload is 209 alerts. We find half the fraud, miss half, and send 172 legitimate transactions for review. These limitations should be stated alongside performance.

Table II applies the three validation cutoffs without retuning on test outcomes. A more permissive policy finds more fraud here, but also creates more false alerts.

The PR curve shows precision against recall across score cutoffs (Fig. 3). Trapezoidal PR AUC connects neighboring points and adds their areas. Its value is 0.1496 on this test period. It is not accuracy or precision at the default cutoff. The test fraud rate is 0.1304%.

For a fixed batch capacity, sort by decreasing anomaly score and review exactly 100 rows. We find 29 frauds and 71 legitimate transactions, so precision@100 is 29%. Tied scores retain chronological order. This is a capacity for the whole test batch, not a daily workload claim.

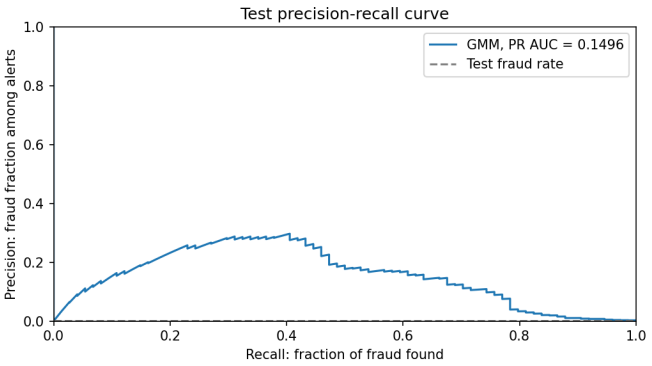


Fig. 3. Final test precision-recall curve. The dashed line marks test fraud prevalence.

VI. Reproducibility, Limits and Conclusion

All analysis code appears directly in `Phase1.ipynb`. The notebook includes its actual outputs, ten explained charts, checks beside the relevant code, and a fixed seed. The requirements file specifies the main package versions. Result tables support this report and the presentation. Each team member must review and understand the AI-assisted work; individual contributions are not inferred from team membership.

The test period contains only 74 frauds. One fraud decision changes recall by about 1.35 percentage points. The short historical dataset, anonymous inputs, unaudited provider PCA, duplicate-removal assumption, diagonal covariance and small K grid limit the conclusion. Future patterns can change, and rarity is not the same as fraud.

The GMM provides a clear Phase 1 review baseline with visible mistakes and review tradeoffs. We do not claim production readiness. Phase 2 may compare an autoencoder using the same legitimate training rows and evaluation protocol; no autoencoder or hybrid results are included here.

References

[1] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. R. Stat. Soc. B*, vol. 39, no. 1, pp. 1–22, 1977, doi: 10.1111/j.2517-6161.1977.tb01600.x.

[2] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[3] scikit-learn, "Gaussian mixture models." [Online]. https://scikit-learn.org/stable/modules/mixture.html.

[4] ULB Machine Learning Group / Worldline, "Credit Card Fraud Detection." [Online]. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud.

[5] TensorFlow, "Classification on imbalanced data," dataset mirror documentation. [Online]. https://www.tensorflow.org/tutorials/structured_data/imbalanced_data. Online sources checked Oct. 21, 2026.